

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr.A.JAYA MABEL RANI

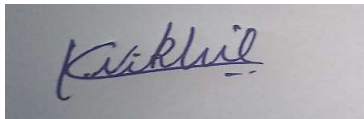
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

*I **K.Nikhil / 192424366** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1. Relational Algebra Operations

Given Relations:

EMPLOYEE(EmpID, EmpName, DeptID, Salary)

DEPARTMENT(DeptID, DeptName)

(a) Selection (σ)

Employees earning more than ₹50,000

$\sigma \text{ Salary} > 50000$ (EMPLOYEE)

(b) Projection (π)

Display employee names only

$\pi \text{ EmpName}$ (EMPLOYEE)

(c) Union ($\cup$)

Employees from two branches

EMPLOYEE_BRANCH1 $\cup$ EMPLOYEE_BRANCH2

(d) Difference ($-$)

Employees not in HR Department

$\pi \text{ EmpID}$ (EMPLOYEE)

$-$

$\pi \text{ EmpID}(\sigma \text{ DeptID}='HR'(\text{EMPLOYEE}))$

(e) Cartesian Product ($\times$)

EMPLOYEE $\times$ DEPARTMENT

(f) Join ($\bowtie$)

Employee names with department names

EMPLOYEE $\bowtie$ EMPLOYEE.DeptID = DEPARTMENT.DeptID DEPARTMENT

Q2. SQL DDL Statements

CREATE TABLE DEPARTMENT

(

DeptID INT PRIMARY KEY,

DeptName VARCHAR(50) NOT NULL UNIQUE

);

CREATE TABLE STUDENT

(

```
SID INT PRIMARY KEY,  
Name VARCHAR(50) NOT NULL,  
DOB DATE NOT NULL,  
DeptID INT,  
FOREIGN KEY (DeptID)  
REFERENCES DEPARTMENT(DeptID)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

Q3. GROUP BY, HAVING and Aggregate Functions

Assume:

SALES(SaleID, Product, Amount)

Count

```
SELECT Product,  
COUNT(*) AS TotalSales  
FROM SALES  
GROUP BY Product;
```

Sum

```
SELECT Product,  
SUM(Amount) AS TotalAmount  
FROM SALES  
GROUP BY Product;
```

Average

```
SELECT Product,  
AVG(Amount) AS AverageSale  
FROM SALES  
GROUP BY Product;
```

Maximum

```
SELECT Product,  
MAX(Amount)  
FROM SALES  
GROUP BY Product;
```

Minimum

```
SELECT Product,  
MIN(Amount)  
FROM SALES  
GROUP BY Product;
```

HAVING

```
SELECT Product,  
SUM(Amount)  
FROM SALES  
GROUP BY Product  
HAVING SUM(Amount) > 100000;
```

Q4. Correlated Subquery

```
SELECT EmpName,  
Salary,  
DeptID  
FROM EMPLOYEE E1  
WHERE Salary >  
(  
SELECT AVG(Salary)  
FROM EMPLOYEE E2  
WHERE E1.DeptID = E2.DeptID  
);
```

Q5. JOIN Operations**INNER JOIN**

```
SELECT *  
FROM EMPLOYEE  
INNER JOIN PROJECT  
ON EMPLOYEE.ProjectID = PROJECT.ProjectID;
```

LEFT OUTER JOIN

```
SELECT *  
FROM EMPLOYEE  
LEFT JOIN PROJECT  
ON EMPLOYEE.ProjectID = PROJECT.ProjectID;
```

RIGHT OUTER JOIN

```
SELECT *  
FROM EMPLOYEE  
RIGHT JOIN PROJECT  
ON EMPLOYEE.ProjectID = PROJECT.ProjectID;
```

FULL OUTER JOIN

```
SELECT *  
FROM EMPLOYEE  
FULL OUTER JOIN PROJECT  
ON EMPLOYEE.ProjectID = PROJECT.ProjectID;
```

CROSS JOIN

```
SELECT *  
FROM EMPLOYEE  
CROSS JOIN PROJECT;
```

Q6. View**Create View**

```
CREATE VIEW DeptSalaryView AS  
SELECT  
D.DeptName,
```

```
COUNT(E.EmpID) AS TotalEmployees,  
AVG(E.Salary) AS AverageSalary  
FROM DEPARTMENT D  
JOIN EMPLOYEE E  
ON D.DeptID = E.DeptID  
GROUP BY D.DeptName;
```

Query on View

```
SELECT *  
FROM DeptSalaryView  
WHERE AverageSalary > 50000;
```

Q7. Relational Algebra

Find students enrolled in **all** courses.

Assume

ENROLLMENT(StudentID, CourseID)

COURSE(CourseID)

Expression using Division:

```
 $\pi$  StudentID  
(  
  ENROLLMENT  $\div$   $\pi$  CourseID(COURSE)  
)
```

Q8. SQL DML Statements

INSERT

```
INSERT INTO STUDENT  
VALUES  
(101,'Rahul','2005-03-12',1);
```

UPDATE

```
UPDATE STUDENT  
SET DeptID = 2  
WHERE SID = 101;
```

DELETE

```
DELETE FROM STUDENT  
WHERE SID = 101;
```

Integrity constraints are enforced through **Primary Key**, **Foreign Key**, and **NOT NULL** constraints.

Q9. Tuple Relational Calculus

Find employees in IT department earning more than ₹50,000.

```
{  
t |  
  EMPLOYEE(t)  
   $\wedge$  t.Dept = 'IT'
```

```
^ t.Salary > 50000
}
```

Q10. Airline Reservation System

Relational Schema

PASSENGER

```
(
  PassengerID,
  PassengerName,
  Age,
  Gender
)
```

FLIGHT

```
(
  FlightID,
  FlightName,
  Source,
  Destination
)
```

BOOKING

```
(
  BookingID,
  PassengerID,
  FlightID,
  JourneyDate,
  SeatNo
)
```

PAYMENT

```
(
  PaymentID,
  BookingID,
  Amount
)
```

Query 1 – INNER JOIN

```
SELECT PassengerName,
  FlightName
FROM PASSENGER
JOIN BOOKING
ON PASSENGER.PassengerID = BOOKING.PassengerID
JOIN FLIGHT
```


ON BOOKING.FlightID = FLIGHT.FlightID;

Query 2 – Subquery

```
SELECT PassengerName
FROM PASSENGER
WHERE PassengerID IN
(
  SELECT PassengerID
  FROM BOOKING
);
```

Query 3 – Aggregate

```
SELECT FlightID,
COUNT(*) AS TotalBookings
FROM BOOKING
GROUP BY FlightID;
```

Query 4 – Create View

```
CREATE VIEW PassengerBooking AS
SELECT PassengerName,
FlightName,
JourneyDate
FROM PASSENGER
JOIN BOOKING
ON PASSENGER.PassengerID = BOOKING.PassengerID
JOIN FLIGHT
ON BOOKING.FlightID = FLIGHT.FlightID;
```

Query 5 – Query on View

```
SELECT *
FROM PassengerBooking
WHERE JourneyDate >= CURRENT_DATE;
```